

Chapter 10: Trees

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives (1 of 2)

- Describe the features of a tree
- Describe various types of tree traversals
- Recognize three common applications where it is appropriate to use a tree
- Describe the features of a binary search tree and the operations on it
- Recognize the three common applications where it is appropriate to use a binary search tree

Learning Objectives (2 of 2)

- Describe the features of an expression tree and the operations on it
- Use an expression tree in recursive descent parsing
- Describe the features of a heap and the operations on it
- Provide a complexity analysis of the heap sort

An Overview of Trees

- In a tree
 - Ideas of predecessor and successor are replaced with **parent** and **child**
- Two main characteristics:
 - Each item can have multiple children
 - All items, except a privileged item called the *root*, have exactly one parent
 - Root has no parent

Tree Terminology (1 of 3)

- Table 10.1 Summary of Terms Used to Describe Trees

Term	Definition
Node	An item stored in a tree
Root	The topmost node in a tree (only node without a parent)
Child	A node immediately below and directly connected to a given node
Parent	A node immediately above and directly connected to a given node
Siblings	The children of a common parent
Leaf	A node that has no children
Interior node	A node that has at least one child

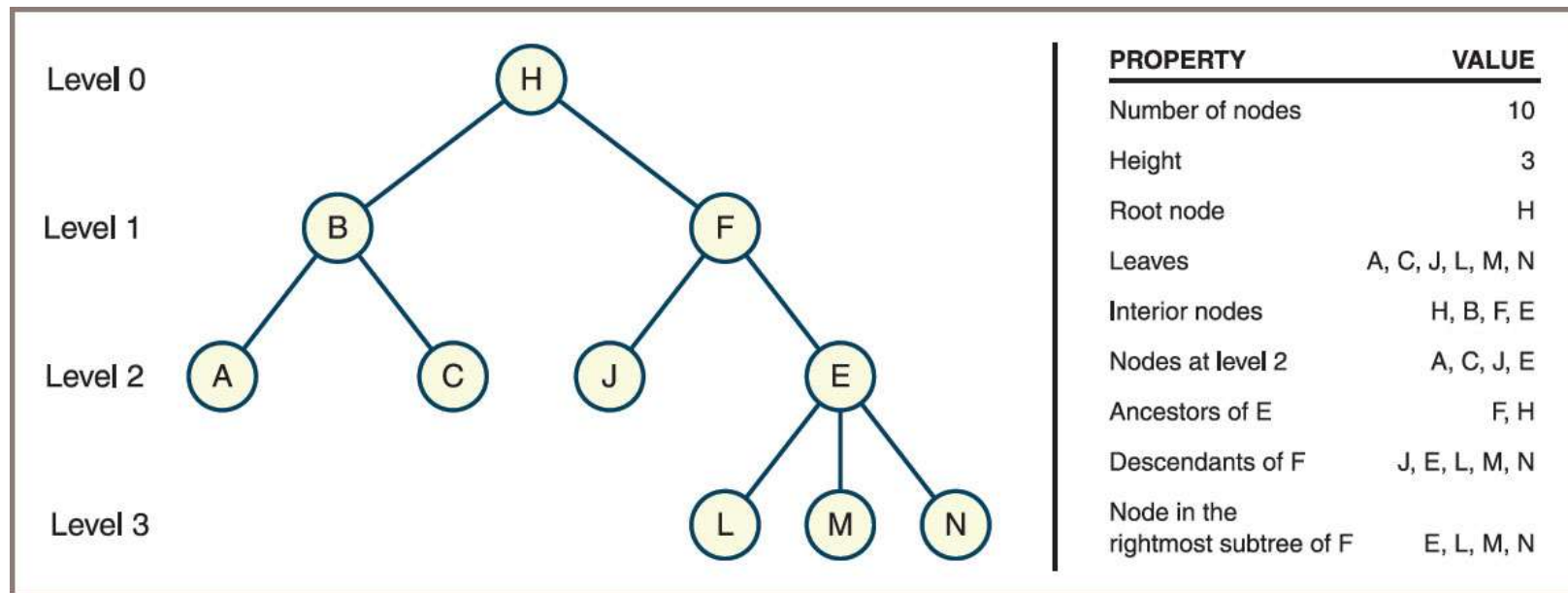
Tree Terminology (2 of 3)

- Table 10.1 Summary of Terms Used to Describe Trees (continued)

Term	Definition
Edge/Branch/Link	The line that connects a parent to its child
Descendant	A node's children, it's children's children, and so on
Ancestor	A node's parent, it's parent's parent, and so on
Path	The sequence of edges that connect a node and one of its descendants
Path length	The number of edges in a path
Depth or level	Equals the length of the path connecting it to the root
Height	The length of the longest path in the tree
Subtree	The tree formed by considering a node and all its descendants

Tree Terminology (3 of 3)

Figure 10-1: Some properties of a tree

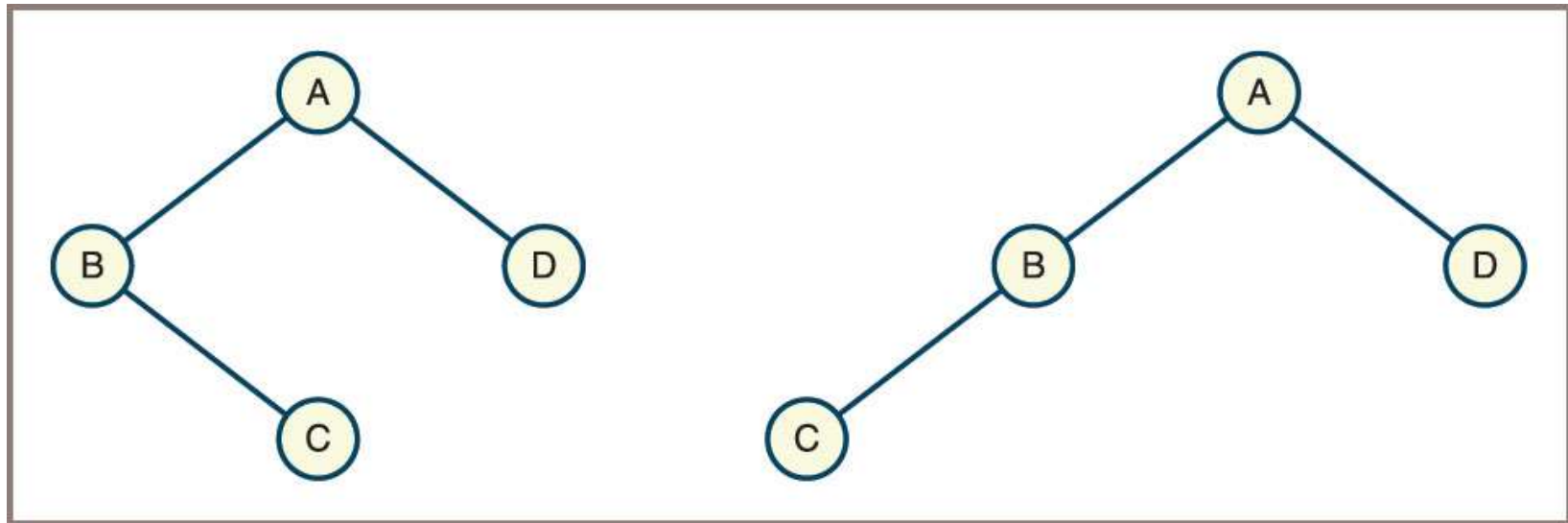


General Trees and Binary Trees (1 of 2)

- General tree is shown in Figure 10.1 (see previous slide)
- Binary tree
 - Each node has at most two children
 - Children are referred to as the left child and the right child
- Two trees shown in Figure 10.2 (next slide)
 - Are not the same when they are considered binary trees
 - Are the same when they are considered general trees

General Trees and Binary Trees (2 of 2)

Figure 10-2: Two unequal binary trees that have the same sets of nodes



Recursive Definitions of Trees

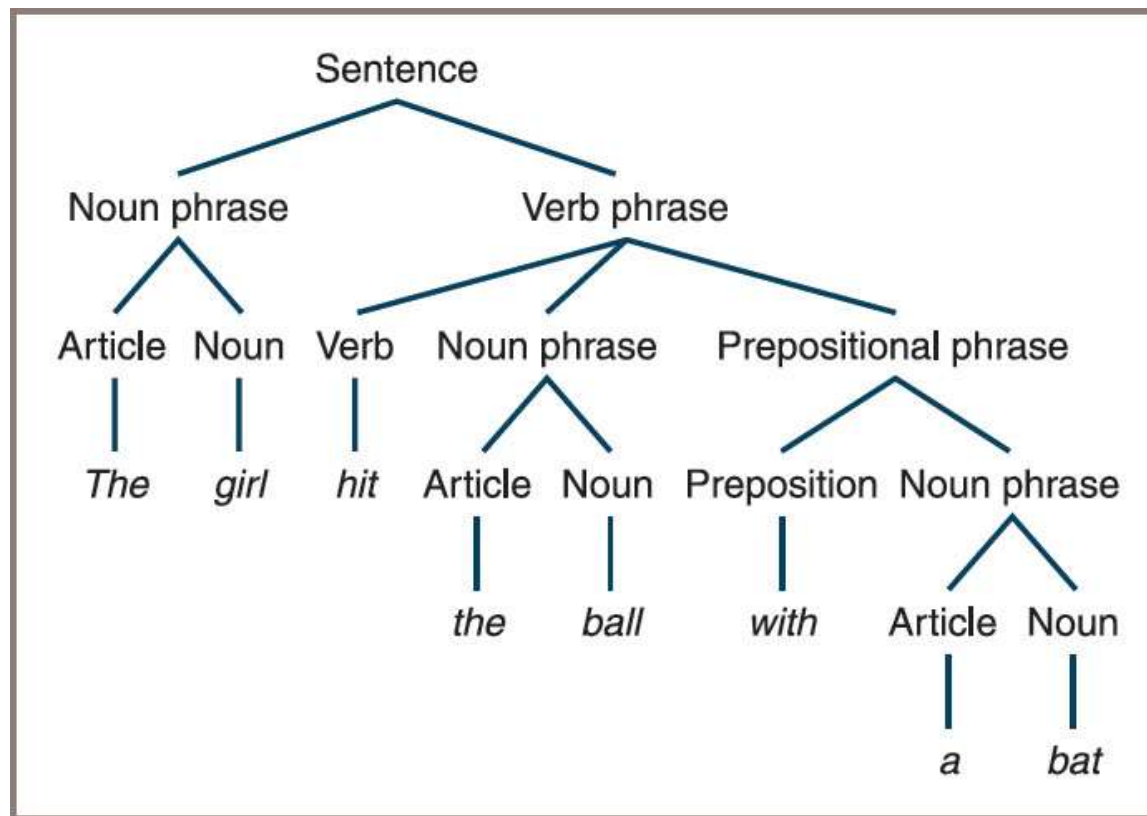
- **General tree**—A general tree is either empty or consists of a finite set of nodes T
 - One node r is distinguished from all others and is called the root
 - In addition, the set $T - \{r\}$ is partitioned into disjointed subsets, each of which is a general tree
- **Binary tree**—A binary tree is either empty or consists of a root plus a left subtree and a right subtree, each of which is a binary tree

Why Use a Tree? (1 of 4)

- Trees nicely represent hierarchical structures
- Parse tree
 - Describes the syntactic structure of a sentence in terms of its components
 - Such as noun phrases and verb phrases
 - See Figure 10.3 on the next slide
- File system structures are also tree-like
 - See Figure 10.4 on 13th slide.

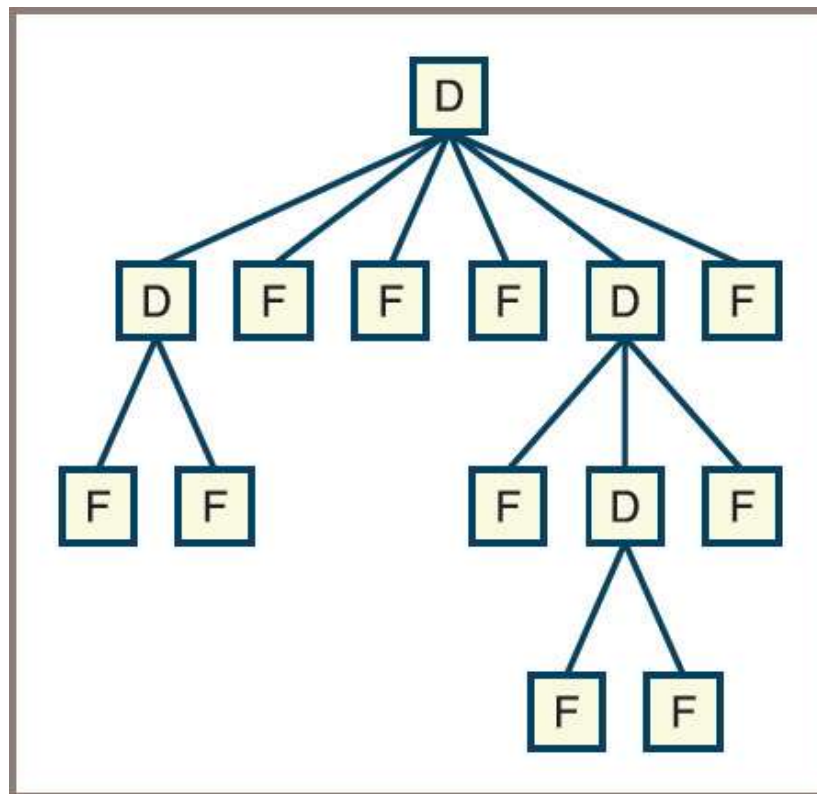
Why Use a Tree? (2 of 4)

Figure 10-3: A parse tree for a sentence



Why Use a Tree? (3 of 4)

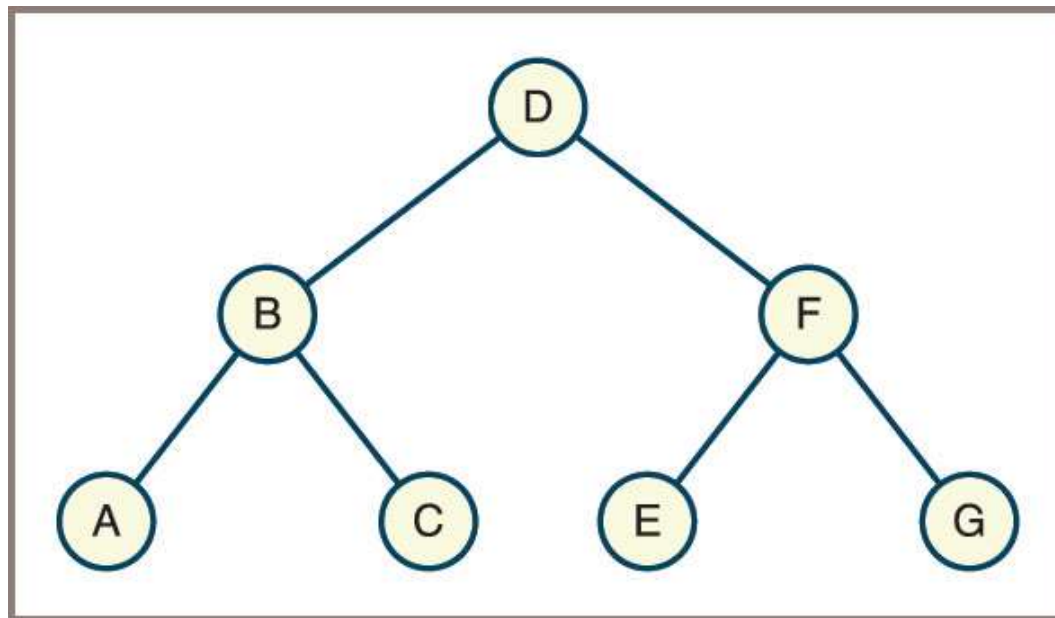
Figure 10-4: A file system structure



Why Use a Tree? (4 of 4)

- Figure 10.5 shows a binary representation of a sorted collection that contains the letters A through G

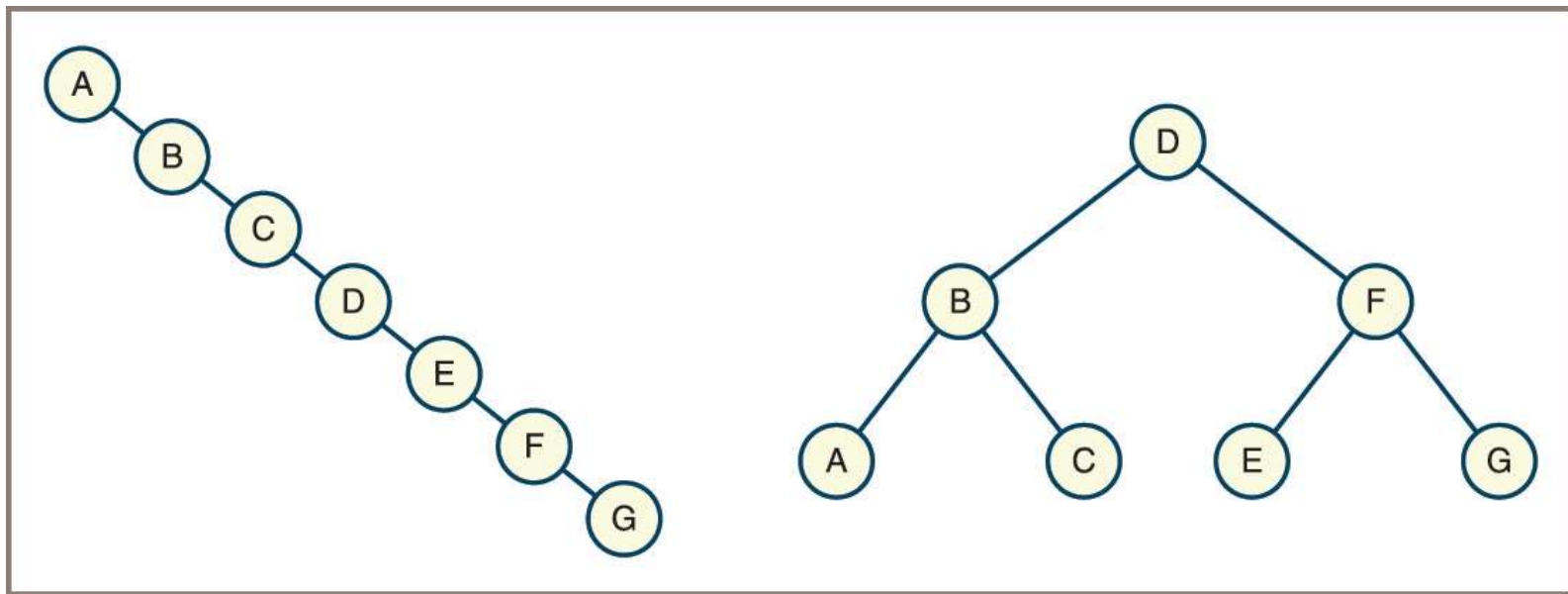
Figure 10-5: A sorted collection as a binary search tree



The Shape of Binary Trees (1 of 5)

- Trees as data structures come in various shapes and sizes
 - Some are vine-like and some are bushy

Figure 10-6: A vine-like tree and a bushy tree



The Shape of Binary Trees (2 of 5)

- The shape of a binary tree can be described by specifying the relationship between its height and the number of nodes it contains
 - A binary tree can be vine-like with N nodes and a height of $N - 1$
 - A full binary tree contains the full complement of nodes at each level

The Shape of Binary Trees (3 of 5)

- Table 10.2 The Relationship Between the Height and the Number of Nodes in Full Binary Tree

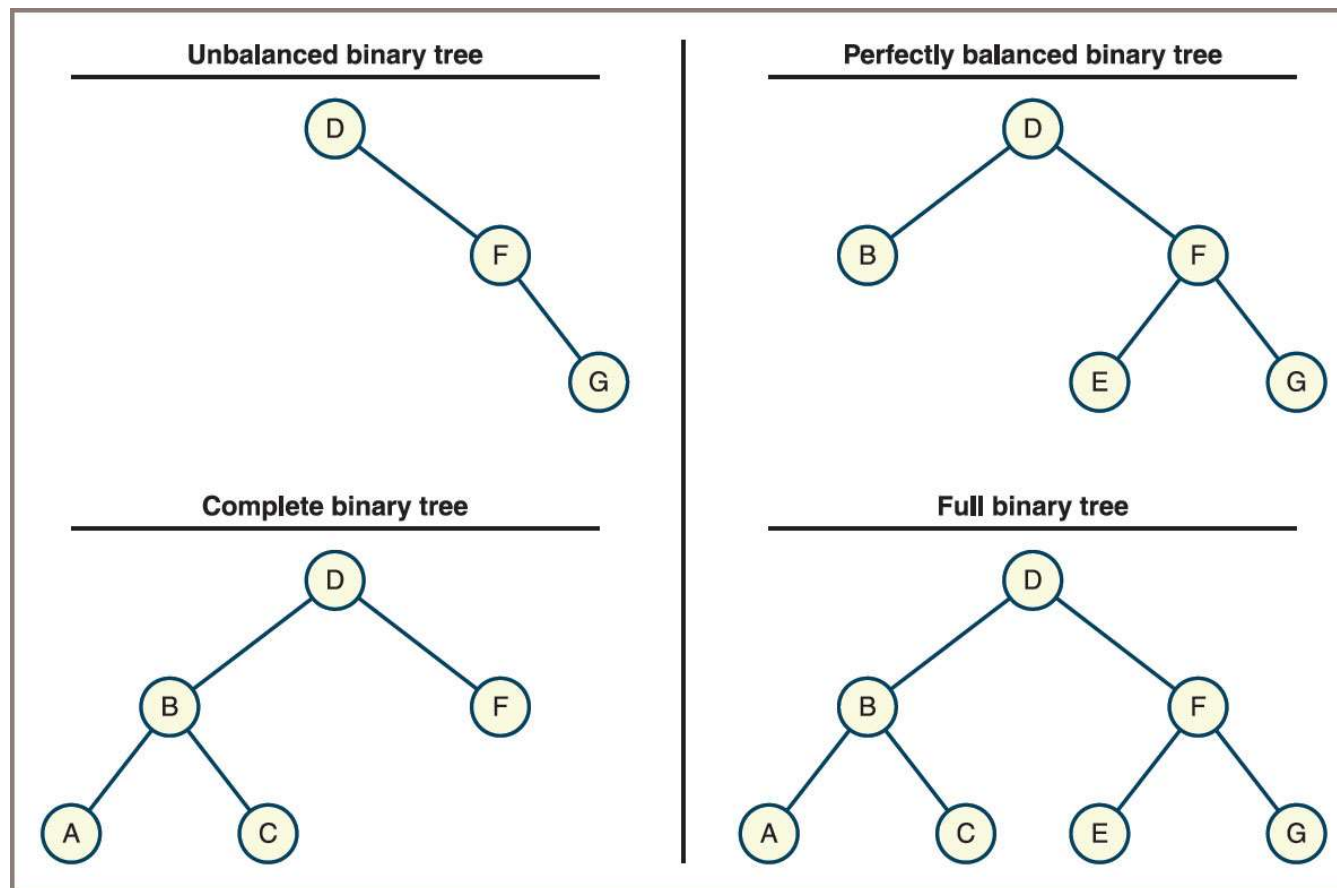
Height of the Tree	Number of Nodes in the Tree
0	1
1	3
2	7
3	15

The Shape of Binary Trees (4 of 5)

- Not all bushy trees are full binary trees
 - However, a *perfectly balanced binary tree*, is bushy enough to support worst-case logarithmic access to leaf nodes
- A **complete binary tree**, in which any nodes on the last level are filled in from left to right, is, like a full binary tree
 - A special case of a perfectly balanced binary tree

The Shape of Binary Trees (5 of 5)

Figure 10-7: Four types of shapes of binary trees



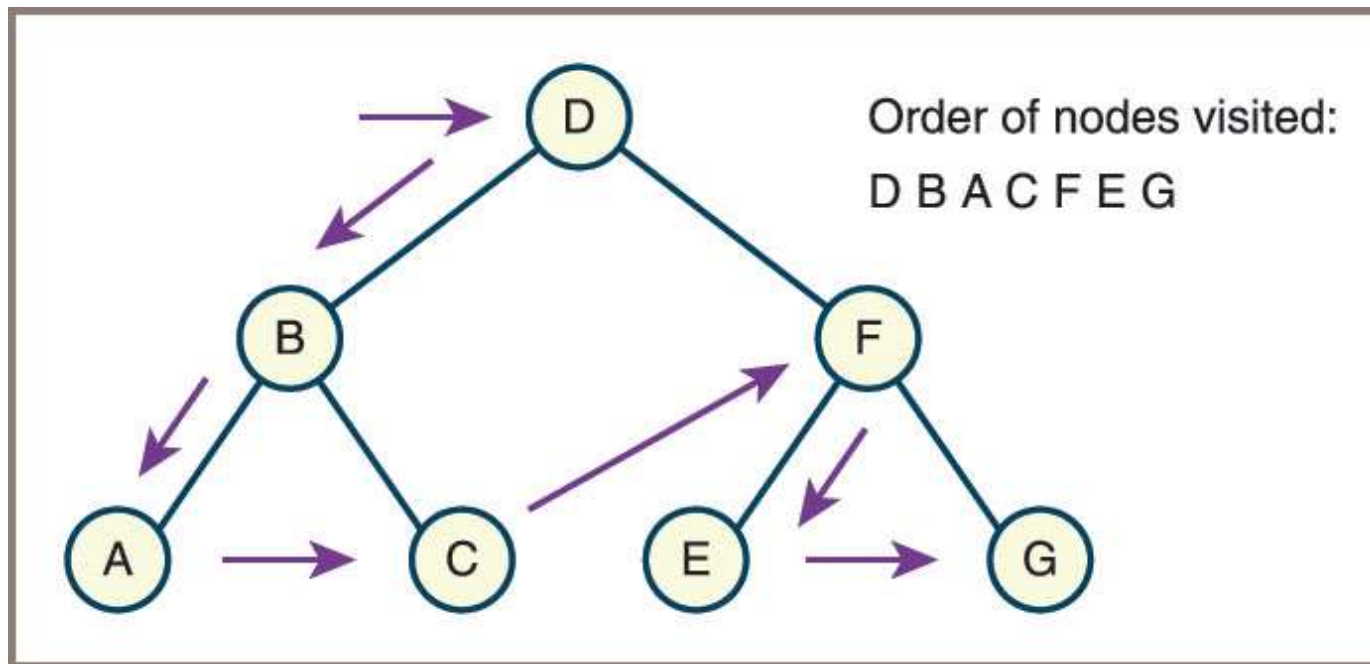
Binary Tree Traversals

- Four standard types of traversals for binary trees:
 - Preorder
 - Inorder
 - Postorder
 - Level order
- Each type follows a particular path and direction as it visits the nodes in the tree

Preorder Traversal

- Preorder traversal algorithm visits a tree's root node and then traverses the left subtree and the right subtree in a similar

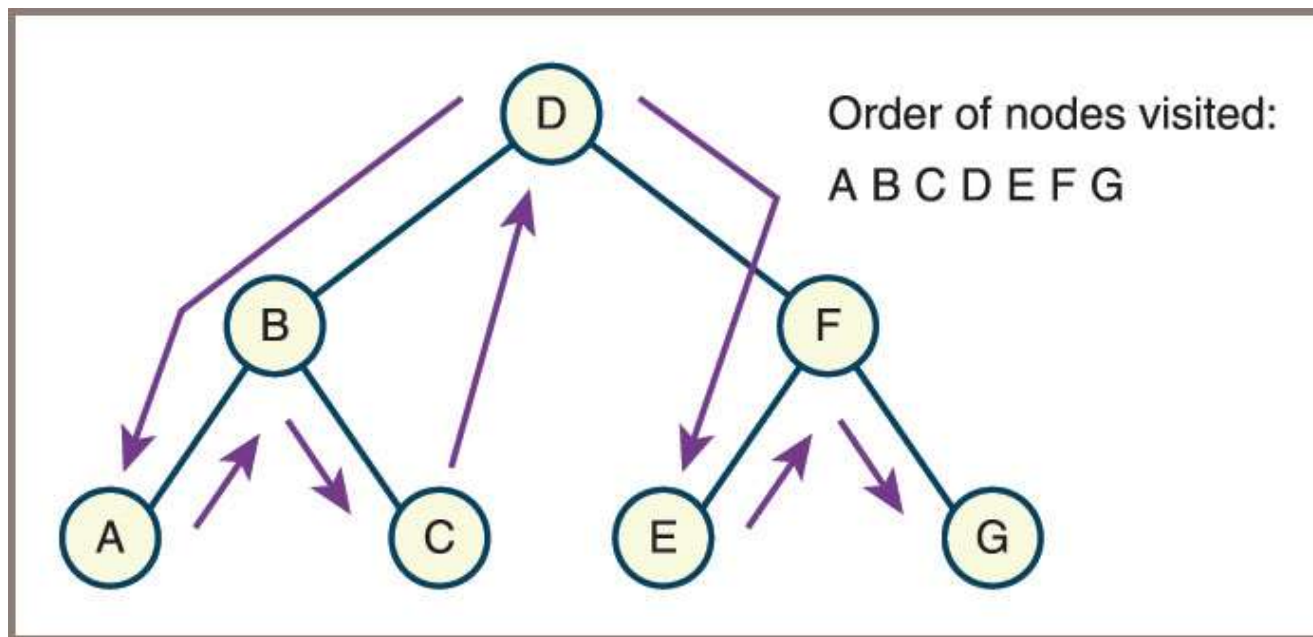
Figure 10-8: A preorder traversal



Inorder Traversal

- Inorder traversal algorithm traverses the left subtree, visits the root node, and traverses the right subtree

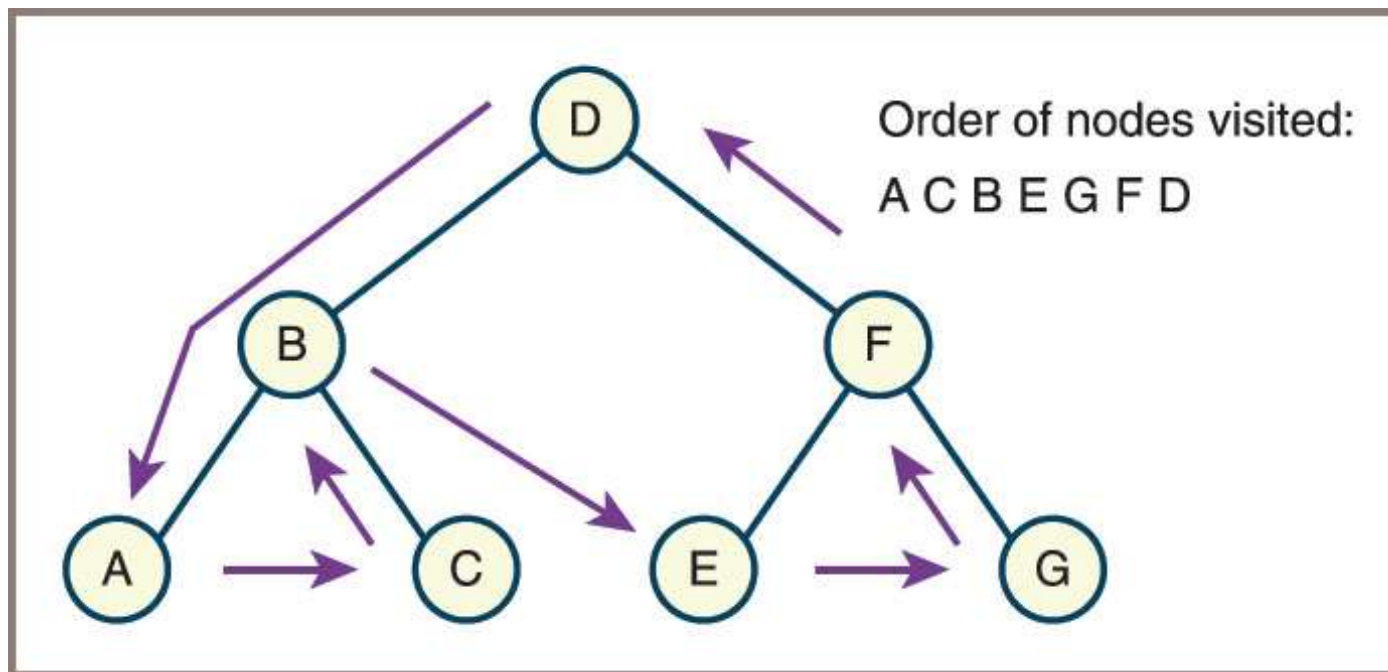
Figure 10-9: An inorder traversal



Postorder Traversal

- Postorder traversal algorithm traverses the left subtree, traverses the right subtree, and visits the root node

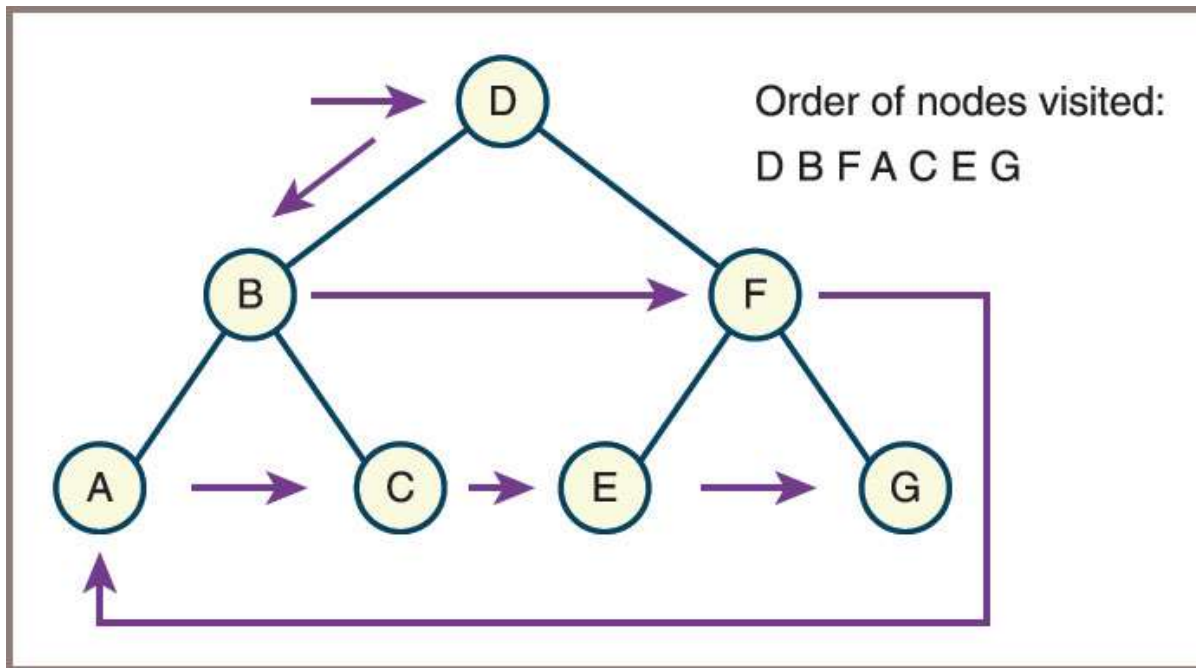
Figure 10-10: A postorder traversal



Level Order Traversal

- Beginning with level 0, this algorithm visits the nodes at each level in left-to-right order

Figure 10-11: A level order traversal



Three Common Applications of Binary Trees

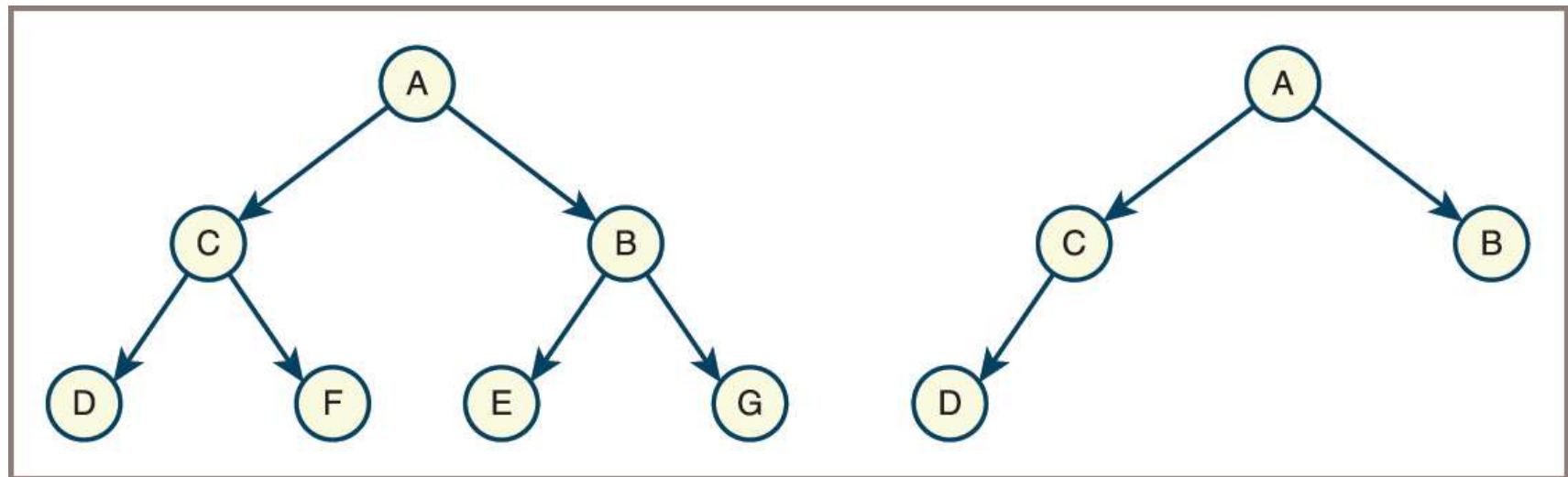
- Heaps
- Binary search trees
- Expression trees

Heaps (1 of 2)

- Min-heap
 - A binary tree in which each node is less than or equal to both of its children
- Max-heap
 - Places the larger nodes nearer to the root
- Either constraint on the order of the nodes is called the heap property
- Heap sort
 - Algorithm that builds a heap from a set of data and then repeatedly removes the root item and adds it to the end of a list
- Heaps are also used to implement priority queues

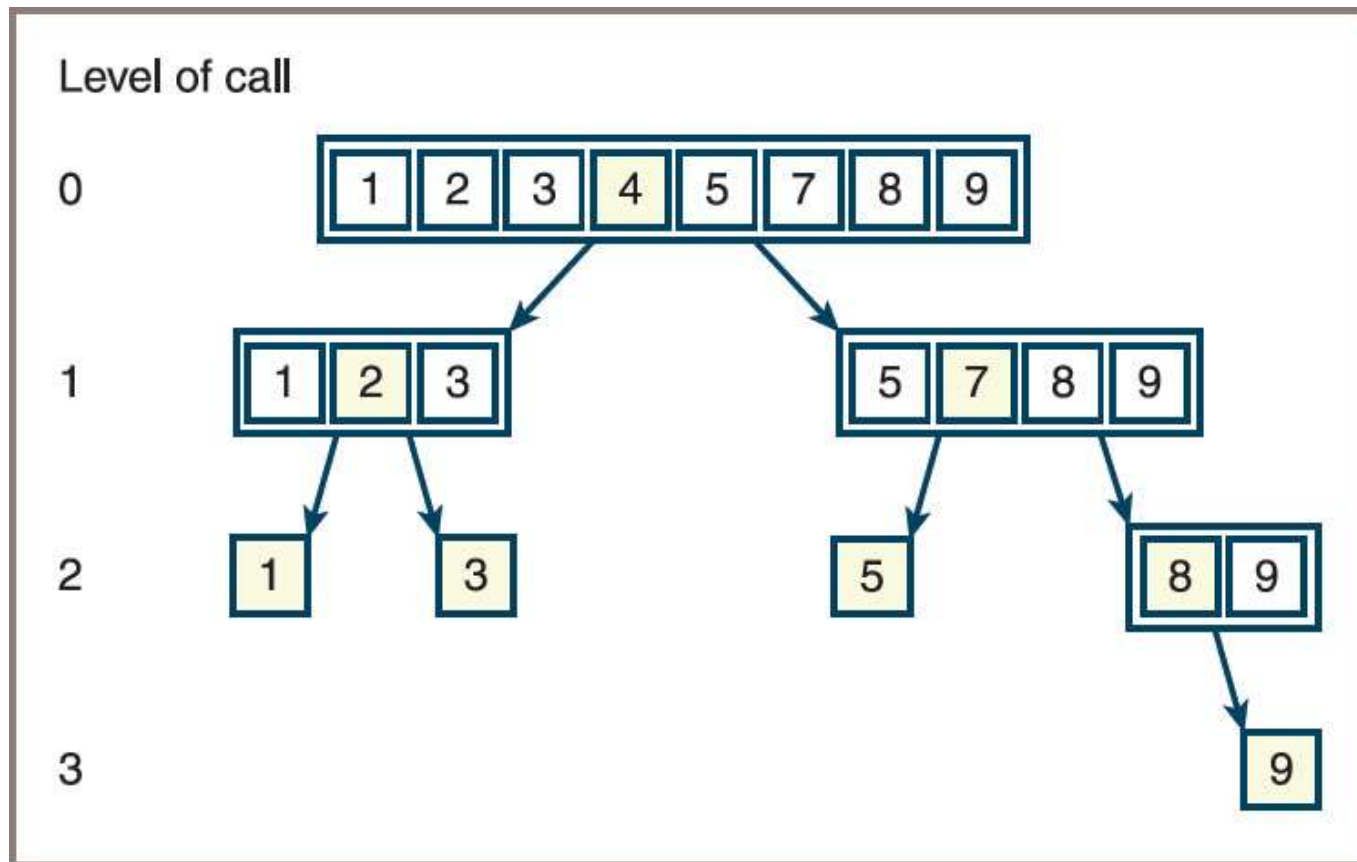
Heaps (2 of 2)

Figure 10-12: Examples of min-heaps



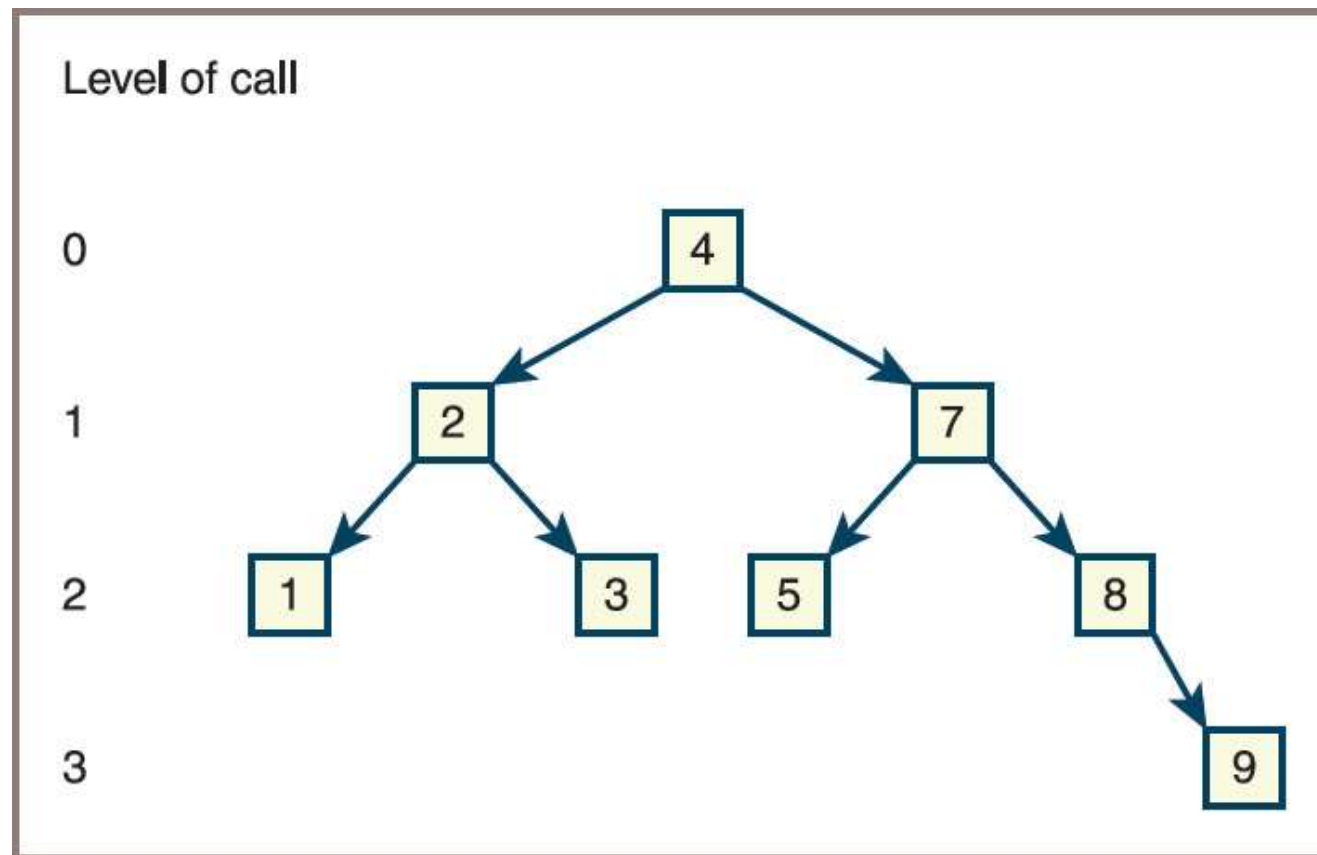
Binary Search Trees (1 of 2)

Figure 10-13: The possible search paths for the binary search of a sorted list



Binary Search Trees (2 of 2)

Figure 10-14: A binary search tree

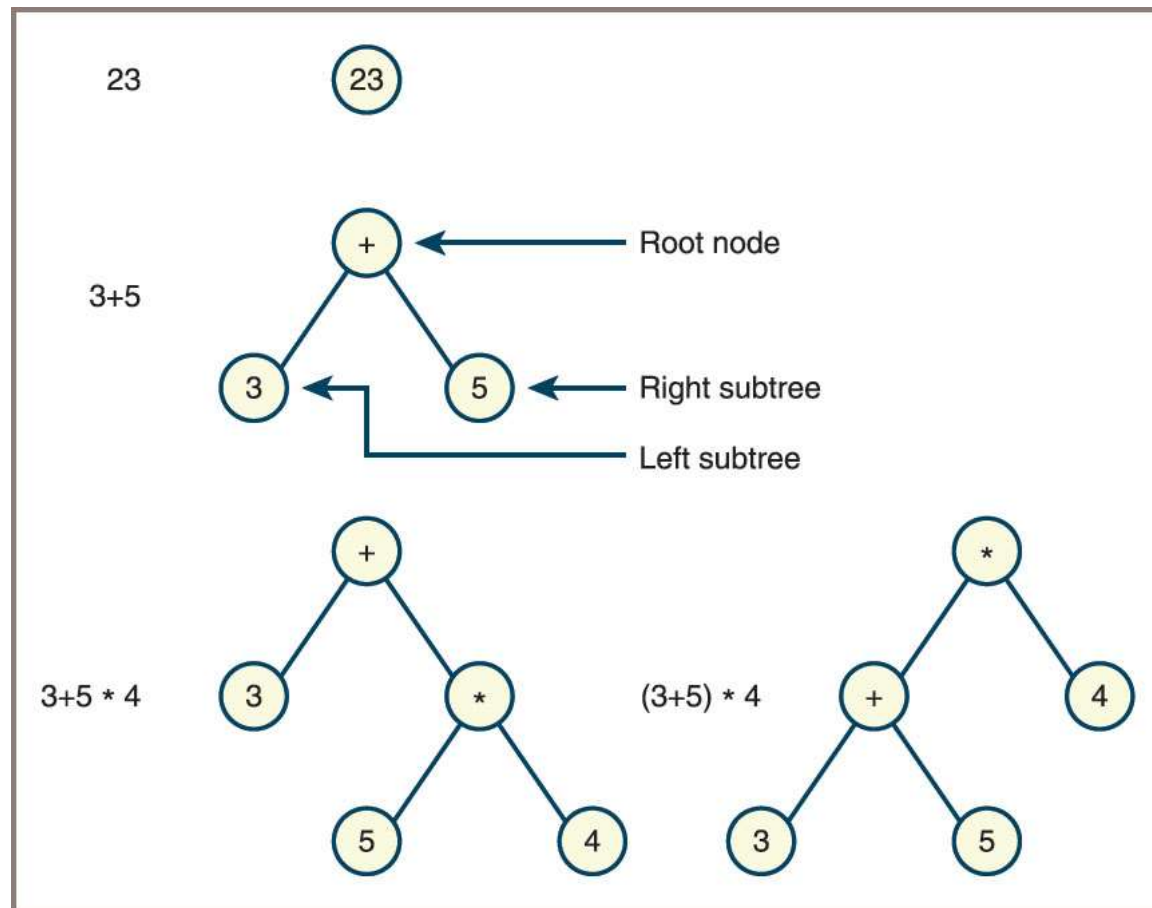


Expression Trees (1 of 2)

- A way to process sentences is to build a parse tree during parsing
- For a language of expressions
 - This structure is also called an expression tree
- Points to remember:
 - An expression tree is never empty
 - Each interior node represents a compound expression, consisting of an operator and its operands
 - Each leaf node represents an atomic, numeric operand
 - Operators of higher precedence usually appear near the bottom of the tree, unless they are overridden in the source expression by parentheses

Expression Trees (2 of 2)

Figure 10-15: Some expression trees



Developing a Binary Search Tree

- A binary search tree imposes a special ordering on the nodes in a binary tree
 - So it supports logarithmic searches and insertions
- This section develops a binary search tree collection and assesses its performance

The Binary Search Tree Interface (1 of 3)

- The interface for a binary search tree should include a constructor and the basic operations common to all collections
 - `isEmpty`, `len`, `str`, `+`, `==`, `in`, `add`, and `count`
- As with bags and sets
 - Insertions and removals are accomplished by the `add` and `remove` methods
- The method `__contains__`
 - Performs a binary search in any BST implementation
- To allow users to retrieve and replace items in a binary search tree, the methods `find` and `replace` are also included
 - The method `find` expects an item as an argument and returns the item matching it in the tree, or `None` otherwise
 - The method `replace` expects two items as arguments

The Binary Search Tree Interface (2 of 3)

- Because there are four ways to traverse a binary tree
 - You include methods for each one
- Each traversal method returns an iterator
 - The tree's `__iter__` method supports a preorder traversal
- Two trees are considered equal if they contain the same items in the same positions
- The `str` operation returns a string that shows the shape of the tree when printed.
- The methods included in any binary search tree class are described in Table 10.3 in the text

The Binary Search Tree Interface (3 of 3)

- The following script creates a BST containing the letters shown in Figure 10.15 and prints its shape:

```
from linkedbst import LinkedBST

tree = LinkedBST()
print("Adding D B A C F E G")
tree.add("D")
tree.add("B")
tree.add("A")
tree.add("C")
tree.add("F")
tree.add("E")
tree.add("G")

# Display the structure of the tree
print("\nTree structure:\n")
print(tree)
```

Data Structure for the Linked Implementation (1 of 7)

- Code for the part of the LinkedBST class that creates a tree:

```
"""
```

```
File: linkedbst.py  
Author: Ken Lambert  
"""
```

```
from abstractCollection import AbstractCollection  
from bstnode import BSTNode
```

```
class LinkedBST (AbstractCollection):  
    """A link-based binary search tree implementation."""  
  
    def __init__(self, sourceCollection = None):  
        """Sets the initial state of self, which includes  
the  
        contents of sourceCollection, if it's present."""  
        self.root = None  
        AbstractCollection.__init__(sourceCollection)  
        # Remaining method definitions go here
```

Data Structure for the Linked Implementation (2 of 7)

- Searching a Binary Search Tree
 - The find method returns the first matching item if the target item is in the tree
 - Otherwise, it returns None
- Pseudocode algorithm for this process:

```
if the tree is empty
    return None
else if the target item equals the root item
    return the root item
else if the target item is less than the root item
    return the result of searching the left subtree
else
    return the result of searching the right subtree
```

Data Structure for the Linked Implementation (3 of 7)

- Traversing a Binary Search Tree
 - General recursive strategy for an inorder traversal of a binary tree:

```
if the tree is not empty
    visit the left subtree
    visit the item at the root of the tree
    visit the right subtree
```

- Code for the recursive implementation of the inorder method:

```
def inorder(self) :
    """Supports an inorder traversal on a view of self."""
    lyst=list()

    def recurse(node) :
        if node != None:
            recurse(node.left)
            lyst.append(node.data)
            recurse(node.right)

    recurse(self.root)
    return iter(lyst)
```

Data Structure for the Linked Implementation (4 of 7)

- The String Representation of a Binary Search Tree
 - Code that builds the appropriate string by first recursing with the right subtree, then visiting an item, and finally recursing with the left subtree:

```
def __str__(self):  
    """Returns a string representation with the tree rotated  
    90 degrees counterclockwise."""  
    def recurse(node, level):  
        s = ""  
        if node != None:  
            s += recurse(node.right, level + 1)  
            s += "| " * level  
            s += str(node.data) + "\n"  
            s += recurse(node.left, level + 1)  
        return s  
    return recurse(self.root, 0)
```

Data Structure for the Linked Implementation (5 of 7)

- Inserting an Item into a Binary Search Tree
 - The add method inserts an item into its proper place in the binary search tree
 - An item's proper place will be in one of three positions:
 - The root node, if the tree is already empty
 - A node in the current node's left subtree, if the new item is less than the item in the current node
 - A node in the current node's right subtree, if the new item is greater than or equal to the item in the current node
 - For the second and third options, the **add** method uses a recursive helper function named **recurse**

Data Structure for the Linked Implementation (6 of 7)

- Code for the **add** method:

```
def add(self, item):
    """Adds item to the tree."""

    # Helper function to search for item's position
    def recurse(node):
        # New item is less; go left until spot is found
        if item < node.data:
            if node.left == None:
                node.left = BSTNode(item)
            else:
                recurse(node.left)
        # New item is greater or equal;
        # go right until spot is found
        elif node.right == None:
            node.right = BSTNode(item)
        else:
            recurse(node.right)
        # End of recurse

    # Tree is empty, so new item goes at the root
    if self.isEmpty():
        self.root = BSTNode(item)
    # Otherwise, search for the item's spot
    else:
        recurse(self.root)
    self.size += 1
```

Data Structure for the Linked Implementation (7 of 7)

- Removing an Item from a Binary Search Tree
 - Save a reference to the root node
 - Locate the node to be removed, its parent, and its parent's reference to this node
 - If the node has a left child and a right child, replace the node's value with the largest value in the left subtree and delete that value's node from the left subtree
 - Otherwise, set the parent's reference to the node to the node's only child
 - Reset the root node to the saved reference
 - Decrement the size and return the item

Complexity Analysis of Binary Search Trees

- Optimal behavior depends on the height of the tree
- Run time of insertions is also highly dependent on the height of the tree
 - Removals also require a search for the target item
- Strategies for maintaining a tree structure that supports optimal insertions and searches in all cases
 - Are the subject of advanced computer science courses

Recursive Descent Parsing and Programming Languages

- This section gives a brief overview of some resources for processing languages
 - Including grammars, parsing, and a recursive descent-parsing strategy

Introduction to Grammars (1 of 4)

- A grammar consists of a few parts:
 - A **vocabulary** (or **dictionary** or **lexicon**) consisting of words and symbols allowed in sentences in the language
 - A set of *syntax rules* that specify how symbols in the language are combined to form sentences
 - A set of **semantic rules** that specify how sentences in the language should be interpreted
 - For example, the statement $\mathbf{x} = \mathbf{y}$ might mean “copy the value of $\mathbf{y}$ to the variable $\mathbf{x}$ ”

Introduction to Grammars (2 of 4)

- Example: suppose you don't want to allow expressions, such as $4 + 3 - 2$ or $4 * 3 / 2$, that contain consecutive adding operations or consecutive multiplying operations
 - The following grammar defines the syntax and vocabulary of this new little language:

```
expression = term [ addingOperator term ]
term = factor [ multiplyOperator factor ]
factor = number | "(" expression ")"
number = digit { digit }
digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" |
"7" | "8" | "9"
addingOperator = "+" | "-"
multiplyingOperator = "*" | "/"
```

Introduction to Grammars (3 of 4)

- This type of grammar is called an Extended Backus-Naur Form (EBNF) grammar
- An EBNF grammar uses three kinds of symbols:
 - **Terminal symbols**—These symbols are in the vocabulary of the language and literally appear in programs in the language—for instance, `+` and `*` in the preceding examples
 - **Nonterminal symbols**—These symbols name phrases in the language, such as `expression` or `factor` in the preceding examples
 - **Metasymbols**—These symbols are used to organize the rules in the grammar

Introduction to Grammars (4 of 4)

- Table 10.4 Metasymbols in EBNF

Metasymbols	Use
"""	Enclose literal items
=	Means “is defined as”
[]	Enclose optional items
{ }	Enclose zero or more items
()	Group together required choices
	Indicates a choice

Recognizing, Parsing, and Interpreting Sentences in a Language

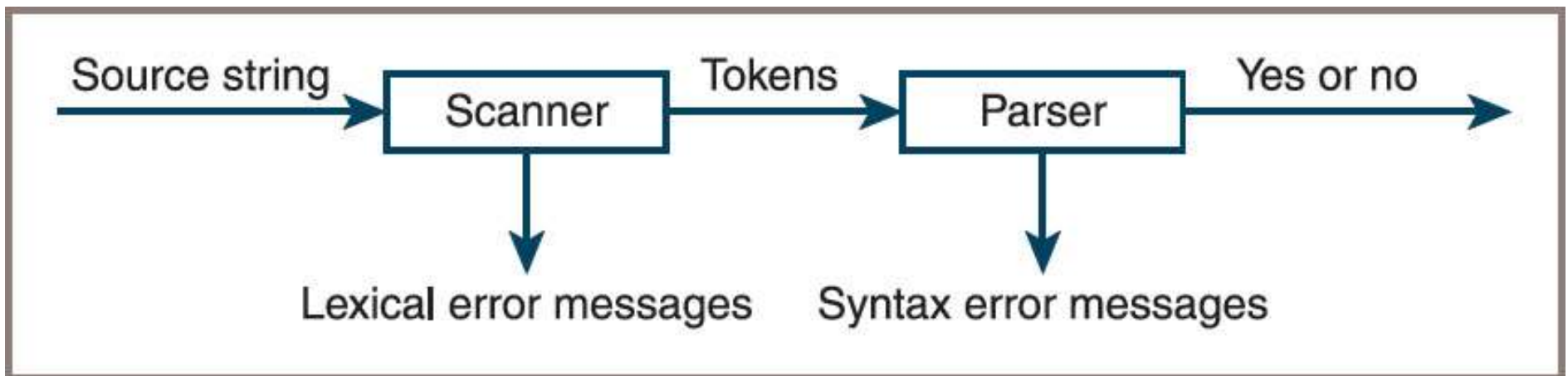
- To process sentences in a language you use:
 - **Recognizer** – analyzes a string to determine if it is a sentence in a given language
 - **Parser** – has all the features of a recognizer and returns information about the syntactic and semantic structure of a sentence
 - **Interpreter** – carries out the actions specified by a sentence

Lexical Analysis and the Scanner (1 of 2)

- When developing a parser
 - It is convenient to assign the task of recognizing symbols in a string to a lower-level module called a *scanner*
- Scanner performs **lexical analysis**
 - In which individual words are picked out of a stream of characters
- Output of the scanner is a stream of words called **tokens**
 - These become the input to a **syntax analyzer** that uses the tokens and the grammar rules to determine whether the program is syntactically correct

Lexical Analysis and the Scanner (2 of 2)

Figure 10-16: A scanner and parser working in tandem

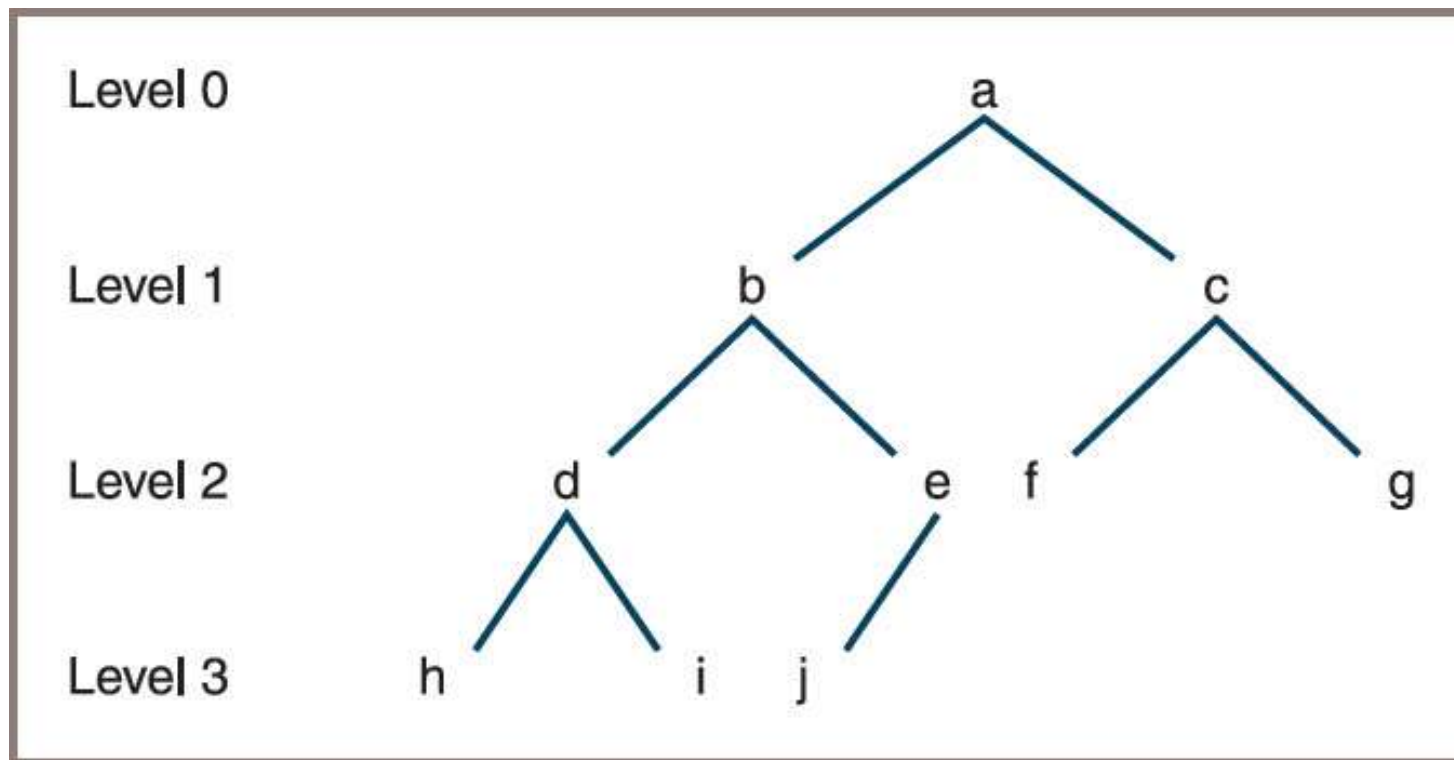


Parsing Strategies

- Recursive descent parsing
 - Defines a function for each rule in the grammar
 - Each function process the phrase or portion of the input sentence covered by its rule
- Note the following:
 - Each nonterminal symbol in the grammar becomes the name of a function in the parser
 - The body of a function processes the phrases on the right side of the rule
 - To process a nonterminal symbol, you simply invoke a function having the symbol's name
 - To process an optional item, you use an **if** statement
 - You observe the current token by calling the method **get** on the scanner object
 - You scan to the next token by calling the method **next** on the scanner object

An Array Implementation of Binary Trees (1 of 4)

Figure 10-18: A complete binary tree



An Array Implementation of Binary Trees (2 of 4)

- In an array-based implementation, the elements are stored by level
 - As shown in Figure 10.19

Figure 10-19: An array representation of a complete binary tree



An Array Implementation of Binary Trees (3 of 4)

- Table 10.6 The Locations of Items in an Array Representation of a Complete Binary Tree

Item	Location
Parent	$(i - 1) / 2$
Left sibling, if there is one	$i - 1$
Right sibling, if there is one	$i + 1$
Left child, if there is one	$i * 2 + 1$
Right child, if there is one	$i * 2 + 2$

An Array Implementation of Binary Trees (4 of 4)

- Thus, for item d at location 3, you get the result shown below
- Table 10.7 The Relatives of Item **d** in an Array Representation of a Complete Binary Tree

Item	Location
Parent	b at 1
Left sibling, if there is one	Not applicable
Right sibling, if there is one	e at 4
Left child, if there is one	h at 7
Right child, if there is one	i at 8

Implementing Heaps (1 of 7)

- Table 10.8 The Methods in the Heap Interface

Method	What It Does
<code>heap.isEmpty()</code>	Returns True if <code>heap</code> is empty or False otherwise.
<code>heap.__len__()</code>	Same as <code>len(heap)</code> . Returns the number of items in <code>heap</code> .
<code>heap.__iter__()</code>	Same as <code>iter(heap)</code> or <code>for item in heap:</code> . Visits the items from least to greatest.
<code>heap.__str__()</code>	Same as <code>str(heap)</code> . Returns a string that shows the shape of the heap.
<code>heap.__contains__(item)</code>	Same as <code>item in heap</code> . Returns True if item is in the heap or False otherwise.

Implementing Heaps (2 of 7)

- Table 10.8 The Methods in the Heap Interface (continued)

Method	What It Does
<code>heap.__add__(otherHeap)</code>	Same as <code>heap + otherHeap</code> . Returns a new heap with the contents of <code>heap</code> and <code>otherHeap</code> .
<code>heap.__eq__(anyObject)</code>	Same as <code>heap == anyObject</code> . Returns True if <code>heap</code> equals <code>anyObject</code> or False otherwise. Two heaps are equal if they contain the same items.
<code>heap.peak()</code>	Returns the topmost item in <code>heap</code> . <i>Precondition:</i> <code>heap</code> is not empty.
<code>heap.add(item)</code>	Inserts <code>item</code> in its proper place in <code>heap</code> .
<code>heap.pop()</code>	Removes and returns the topmost item in <code>heap</code> . <i>Precondition:</i> <code>heap</code> is not empty.

Implementing Heaps (3 of 7)

- The two most critical heap operations are:
 - **add** method – expects a comparable element as an argument and inserts the element into its proper place in the heap
 - **pop** method – deletes the topmost node in the heap, returns the element contained there, and maintains the heap property
- The **peek** operation returns but does not remove the topmost element in a heap

Implementing Heaps (4 of 7)

- Code for the `add` method:

```
def add(self, item):
    self.size += 1
    self.heap.append(item)
    curPos = len(self.heap) - 1
    while curPos > 0:
        parent = (curPos - 1) // 2           # Integer quotient!
        parentItem = self.heap[parent]
        if parentItem <= item:               # Found the spot
            break
        else:                               # Continue walking up
            self.heap[curPos] = self.heap[parent]
            self.heap[parent] = item
            curPos = parent
```

Implementing Heaps (5 of 7)

- Strategy for removals:
 - Begin by saving pointers to the top element and the bottom element in the heap and by moving the element from the bottom of the heap to the top
 - Walk down the heap from the top, moving the smallest child up one level, until the bottom of the heap is reached

Implementing Heaps (6 of 7)

- Code for the pop method:

```
def pop(self):
    if self.isEmpty():
        raise AttributeError("Heap is empty")
    self.size -= 1
    topItem = self.heap[0]
    bottomItem = self.heap.pop(len(self.heap) - 1)
    if self.isEmpty():
        return bottomItem
    self.heap[0] = bottomItem
    lastIndex = len(self.heap) - 1
    curPos = 0
    while True:
        leftChild = 2 * curPos + 1
        rightChild = 2 * curPos + 2
        if leftChild > lastIndex:
            break
```

Implementing Heaps (7 of 7)

- Code for the pop method: (continued)

```
if rightChild > lastIndex:
    maxChild = leftChild
else:
    leftItem = self.heap[leftChild]
    rightItem = self.heap[rightChild]
    if leftItem < rightItem:
        maxChild = leftChild
    else:
        maxChild = rightChild
    maxItem = self.heap[maxChild]
    if bottomItem <= maxItem:
        break
    else:
        self.heap[curPos] = self.heap[maxChild]
        self.heap[maxChild] = bottomItem
        curPos = maxChild
return topItem
```

Chapter Summary (1 of 2)

- Trees are hierarchical collections
 - The topmost node in a tree is called its root
- In a binary tree, a node can have at most two children
- There are four standard types of tree traversals: preorder, inorder, postorder, and level order
- An expression tree is a type of binary tree in which the interior nodes contain operators and the successor nodes contain their operands
- A binary search tree is a type of binary tree in which each nonempty left subtree contains data that are less than the datum in its parent node
 - Each nonempty right subtree contains data that are greater than the datum in its parent node

Chapter Summary (2 of 2)

- A binary search tree supports logarithmic searches and insertions if it is close to complete
- A heap is a type of binary tree in which smaller data items are located near the root
 - You can use a heap to implement the $n \log n$ heap sort algorithm and a priority queue